EduSmart School ERP ��? Backend Roadmap

EduSmart School ERP — Backend Roadmap

> The frontend architecture (Flutter, feature-first + Riverpod) has already been designed. This document is its backend counterpart — every module currently runs on dummy data, and this is the step-by-step plan to connect it to a real backend.

---

1. Scope — Real Entities Extracted from the Codebase

Every module's models were scanned — these are the core domains that need to exist on the backend (the dummy-data classes are already shaped this way and can be used as a direct reference):

Module	Core Entities (already present in Flutter models)
Students	Student, StudentStatus, RiskLevel
Teachers	Teacher, Department
Attendance	AttendanceRecord, AttendanceStatus, AttendanceTrendPoint
Fees	FeeHead, Installment, FeeTransaction, ExpenseEntry, IncomeEntry, AuditLogEntry
Examination	Examination, ExamSubjectSchedule, QuestionItem, QuestionPaperSet,

Module	Core Entities (already present in Flutter models)
	EvaluationRecord, ResultRecord, RevaluationRequest, MalpracticeIncident, SeatAssignment, Invigilator
HR & Payroll	Employee, Department, Designation, LeaveRequest, Shift, PayrollRecord, SalaryStructure, LoanAdvance, TaxRecord, JobOpening, Candidate
Transport	TripRow, RouteSummaryRow, AlertData
Hostel	HostelBlock, Room, Bed, StaffMember, Visitor, Complaint, MaintenanceRequest, LeaveRequest, Payment
Inventory	InventoryItem, StockActivity, CategoryData
Parents (portal)	ChildRecord, FeeSummary, AcademicSummary, HomeworkItem, TimetableSlot, TransportInfo, LibraryBook, ChatMessage
Settings	App-level config

Observation: The `Student` class is already duplicated across multiple modules in different shapes (fees, examination, hostel, students, parents) — on the backend this must become **one single Student table/entity** that every module references via foreign key, so the data stays consistent.

---

2. Tech Stack Recommendation

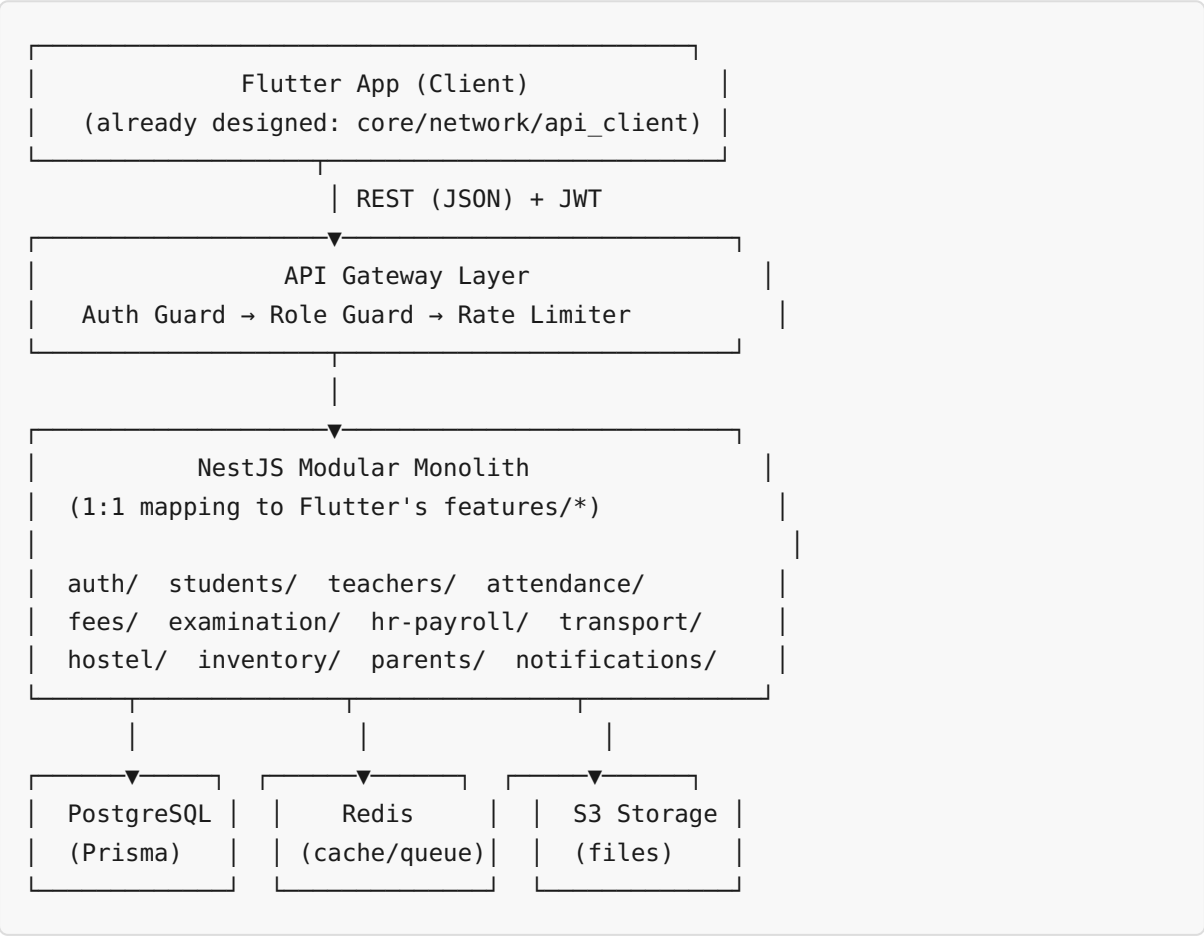
Layer	Recommendation	Reasoning
API Framework	**NestJS (Node.js + TypeScript)**	Modular architecture (modules/controllers/ services) maps directly onto Flutter's feature-first structure. Strong typing,

Layer	Recommendation	Reasoning
		built-in DI, large ecosystem.
Database	**PostgreSQL**	Fees, payroll, exam results — all strongly relational data (foreign keys, transactions are essential). A relational DB is the best fit.
ORM	**Prisma**	Type-safe queries, built-in migrations, integrates well with NestJS.
Auth	**JWT (access + refresh token)** + role-based guards	The Flutter models already have `UserRole` / `AppRole` / `Permission` enums — these become the base for backend RBAC.
File Storage	**S3-compatible storage** (AWS S3 / Cloudflare R2 / MinIO for self-hosting)	PDF receipts, question papers, ID cards, documents — all stored here.
Caching/Queue	**Redis**	Session cache, rate-limiting, background jobs (payroll processing, bulk notifications).
Background Jobs	**BullMQ** (Redis-based)	Payroll batch processing, bulk SMS/email/ notifications, PDF generation — heavy/async tasks.
Real-time (optional, later)	**WebSockets (Socket.io)**	Parent-teacher chat, live notifications — Phase 2+.
API Docs	**Swagger/OpenAPI** (built into NestJS)	Auto-generated API reference for the frontend team.
Hosting	Railway / Render / AWS (ECS+RDS) — any, containerized (Docker)	Scalable, environment-based deployment.

> If the team is already comfortable with Python/Django or Firebase, those are viable options too — Firebase Firestore (NoSQL) is good for fast prototyping, but for complex relational reports like fees/payroll/exams, Postgres will be more reliable long-term.

---

3. High-Level Backend Architecture



Why a “Modular Monolith” instead of microservices? Given the current scale/team size, one deployable app makes sense — keeping strict module boundaries (as on the Flutter side too) means it will still be easy to extract individual modules into microservices later if needed.

---

4. Backend Module ↔ Flutter Feature Mapping

Every backend module maps exactly to a Flutter `features/<module>` — so both the frontend and backend teams work with the same mental model:

```
src/
├── auth/                   # login, refresh token, role/permission
├── students/               # single source of truth — every module references
this
├── teachers/
├── attendance/             # for both students and staff
├── fees/                   # fee-heads, installments, transactions, expense/
income, audit-log
├── examination/            # exams, question-bank, seating, evaluation,
results, revaluation
├── hr-payroll/             # employees, payroll, leave, recruitment
├── transport/              # routes, trips, alerts
├── hostel/                 # blocks, rooms, beds, complaints, visitors
├── inventory/              # items, stock activity, categories
├── parents/                # aggregation layer — combined view of students/
fees/attendance/examination per child
├── notifications/          # shared — every module sends notifications through
here
└── common/                 # shared DTOs, guards, interceptors, filters
```

---

5. Auth & Role Design (RBAC)

The Flutter code already has 3 separate role enums (`UserRole` in fees, `AppRole` / `Permission` in examination) — on the backend these need to be consolidated into **one unified role system**:

Unified Roles: `super_admin` , `admin` , `principal` , `teacher` , `accountant` , `cashier` , `hr_staff` , `hostel_warden` , `transport_manager` , `inventory_manager` , `parent` , `student`

Design: - One `User` table (login credentials) → linked to one `Role` + optionally linked to a `Student` / `Employee` / `Parent` record (the actual person this user corresponds to) - Permission granularity: module-level (`fees:read` , `fees:write` , `examination:approve_results` , etc.) — generalize the pattern already used by the examination module's `Permission` enum - JWT payload: `userId` , `role` , `permissions[]` - Every backend module's controller endpoints are guarded with `@Roles()` / `@Permissions()` decorators

6. API Design Conventions

Aspect	Convention
Style	REST, JSON
Base path	`/api/v1/<module>/<resource>`
Auth	`Authorization: Bearer <JWT>`
Pagination	`?page=1&limit=20` → response returns `{ data, total, page, totalPages }`
Filtering	Query params (`?status=active&class=10`)
Error format	`{ statusCode, message, error, timestamp, path }` (consistent across all modules)
Versioning	URL-based (`/v1/`) — `/v2/` for future breaking changes
Soft delete	Most entities (Student, Employee, InventoryItem) use a `deletedAt` field instead of hard-delete

7. Phase-wise Backend Roadmap

Phase B0 — Foundation (1-2 weeks)

- NestJS project setup (modular structure, ESLint/Prettier)
- PostgreSQL + Prisma setup, connect
- `common/` — global exception filter, response interceptor, validation pipe (`class-validator`)
- Docker + docker-compose (app + postgres + redis for local dev)
- CI pipeline (lint + test on PR)

Phase B1 — Auth + Core Identity (1-2 weeks)

- `auth` module: register/login/refresh-token/logout
- Unified `User`, `Role`, `Permission` schema
- `students` module: Student CRUD (single source of truth — every module will reference this)
- `teachers` / `employees` base schema

Phase B2 — High-Value Modules (3-4 weeks)

Same priority as in the frontend roadmap: - **fees**: fee-heads, installments, transactions, receipts (PDF generation), reports - **attendance**: mark/bulk-import/export, trends - **examination**: exam scheduling, results — core flow first (schedule → marks entry → result declaration), advanced features (revaluation, malpractice) later

Phase B3 — Remaining Modules (4-6 weeks)

- `hr-payroll`, `transport`, `hostel`, `inventory` — one at a time, same pattern (module → schema → CRUD → business logic → tests)

Phase B4 — Parent Portal Aggregation (1-2 weeks)

- `parents` module — no new data, just a combined read-only view of `students` / `fees` / `attendance` / `examination` for a child
- Multi-child support (a parent with multiple children)

Phase B5 — Cross-cutting Features (2-3 weeks)

- `notifications` module (in-app + push hook, email/SMS provider integration)
- File upload/storage (S3) — receipts, ID cards, question papers, documents
- Background jobs (payroll batch processing, bulk notifications, report generation)
- Audit logging — shared `AuditLogEntry` for all modules (the fees module already has this concept)

Phase B6 — Hardening & Launch (2-3 weeks)

- Rate limiting, input sanitization, security headers
- Load testing critical endpoints (fee payment, bulk attendance marking, result declaration)
- Finalize API documentation (Swagger)
- Staging → Production deployment pipeline
- Monitoring/alerting (e.g. Sentry for errors, uptime monitoring)

---

8. Step-by-Step Execution Checklist

1. ☐ **Decide: single-school or multi-school (SaaS)?** — If multi-school, add `schoolId` to every table from Day 1 (see Section 9 — this is the most critical decision; don't write any schema before this).
2. ☐ Repo setup — NestJS monorepo (or standalone) + Docker + Postgres + Redis local environment
3. ☐ `common/` — exception filter, response interceptor, validation pipe, logger
4. ☐ Prisma schema — `User`, `Role`, `Permission` (unified auth base)
5. ☐ `auth` module — register/login/JWT refresh/logout endpoints + guards
6. ☐ Prisma schema — `Student` (single source of truth entity)
7. ☐ `students` module — CRUD + role-guarded endpoints
8. ☐ Prisma schema — `Employee`, `Department`, `Designation` (shared by teachers + hr-payroll)
9. ☐ `teachers` module — CRUD
10. ☐ Prisma schema — `FeeHead`, `Installment`, `FeeTransaction`, `ExpenseEntry`, `IncomeEntry`
11. ☐ `fees` module — collection, receipt PDF (S3 upload), reports endpoints

12. ☐ Prisma schema — `AttendanceRecord` (make it generic for both students and staff)
13. ☐ `attendance` module — mark/bulk-import/export/trends endpoints
14. ☐ Prisma schema — Examination domain (Exam, Schedule, QuestionBank, Evaluation, Result)
15. ☐ `examination` module — core flow (schedule → marks → result) first, advanced flows later
16. ☐ **Anonymous evaluation system** — encrypted anonymous-ID↔student mapping table + QR/barcode answer-book lifecycle + `revealIdentity` permission-gated + audit-logged action (see Section 9)
17. ☐ Remaining modules (`hr-payroll` , `transport` , `hostel` , `inventory`) — repeat the same pattern
18. ☐ `parents` aggregation module — read-only combined views
19. ☐ `notifications` module — normal queued channel + **separate high-priority/real-time channel for hostel SOS alerts** (see Section 9)
20. ☐ S3 file service + background job queue (Redis/BullMQ)
21. ☐ Security hardening pass (rate limiting, sanitization, headers) + load test critical endpoints
22. ☐ Finalize Swagger docs + staging deploy
23. ☐ Production deploy + monitoring setup

---

9. Domain-Specific Gaps (Audit Round 2)

Digging deeper into the codebase surfaced 4 important things that go beyond generic CRUD — these need to be explicitly planned for in the roadmap:

Gap	What was found in the codebase	Backend implication
Anonymous evaluation / blind grading security	`examination/services/security_services.dart` already has a proper `RbacService` (a fine-grained permission matrix — very detailed, with `examController` / `examCoordinator` roles each holding 20+ distinct permissions) plus an `AnonymousBook` system that uses `sha256` + `Random.secure()` to generate non-sequential anonymous IDs so evaluators can't see student identity.	This isn't just UI logic — it's **security-critical** on the backend. The anonymous-ID↔real-student mapping table must be **encrypted at rest**, and a "reveal identity" action must be gated behind a specific permission (`revealIdentity`) and audit-logged. This can't be treated like normal CRUD — it needs to be a dedicated sub-task within Phase B2 (examination).
QR/Barcode generation	`barcode_center_screen.dart` uses `qr_flutter` to generate answer-book QR codes, with tracking (assigned/used/unused counts).	The backend needs to handle QR data encode/decode plus the answer-book lifecycle status (generated → assigned → submitted → evaluated) — this is its own sub-domain within `examination` and should have been called out explicitly in Section 7 Phase B2.
Multi-tenancy / single-school assumption	No `school_id` or `tenant_id` field exists anywhere in the codebase — meaning this is currently a **single-school** app.	**A critical decision that was missing from the roadmap:** is this for one school only, or will it eventually serve multiple schools (a SaaS product)? If multi-school is the plan, `schoolId` needs to be added to every table **from Day 1** — retrofitting it later is very costly (both migration and data isolation). This decision

Gap	What was found in the codebase	Backend implication
		needs to be made before Phase B0 starts.
Real-time / urgent alerts	The hostel module's `emergency_sos_card.dart` — Fire/Medical/Security alert buttons that need to reach the warden/security **instantly**.	This is different from the normal `notifications` module (Phase B5, queued/async) — an SOS-type alert needs a **real-time push** (WebSocket or a high-priority FCM channel), not something delay-tolerant. The roadmap should explicitly add a "priority/urgent channel" inside the `notifications` module.
Cross-module search	`fees/features/search/command_palette.dart` searches students and fee-transactions together (command-palette style).	If this kind of global search is eventually needed across all modules (students, employees, inventory), a simple `LIKE` query won't scale — consider **Postgres full-text search** (`tsvector`) or, later, Elasticsearch/ Meilisearch. Worth noting in Phase B5/B6.

Most important open question: the multi-tenancy decision (single school vs. multi-school SaaS) changes the entire DB schema design — it needs to be confirmed before Phase B0 begins.

---

10. Frontend-Backend Sync Point

Whenever a backend module is ready, the Flutter side's `core/network/api_client.dart` (already designed) is used to switch that module's `RepositoryImpl` from dummy data to a real API call — the rest of the UI/

Provider code stays exactly the same (this is exactly what the Data Layer design in the earlier architecture document, Section 7, was built for).

Next step: let me know — should I scaffold an actual NestJS + Prisma project (folder structure + starter code) starting from Phase B0, or write the full Prisma schema + API endpoint design for one module first (e.g. `fees` *or* `students`*)?*